

JavaScript Closures - Complete Guide

What is a Closure?

- A **closure** is a powerful JavaScript feature where an inner function has access to variables from its outer (enclosing) function's scope, even after the outer function has finished executing and its execution context has been destroyed.
- A closure is a memory block that stores the state(variables) and behaviour(functions) of a parent function, so that a child function can access them even after the parent function finished executing.
- Closures are created automatically when a function is defined inside another function and references variables from the outer function then the JIT compiler senses this before touching the inner function and they create a closure.
- It is powerful for preserving data without using global variables or classes(because classes are functions in JavaScript).
- The child function keeps a reference to the outer function's variables -- not a copy -- so the values remain live and can be updated. **Real life example** - Imagine a parent owns a plot of land (variables and a logic). When a parent gives parts of the land to their children(functions), each child carries their portion and the tools(behaviour) needed to manage it. Even if the parent is no longer around, the child still owns and works their land. This is how closures preserve data and behaviour.

Formal Definition

A closure is the combination of a function and the lexical environment within which that function was declared.

Understanding Closures Through Examples

Basic Closure Example

```
function outer() {
  console.log("Inside Outer function");
  const userId = "123";
  const password = "abc123";
  const inner = () => {
    console.log("Inside inner function");
    console.log("User Id:", userId);    // Accessing outer function's variable
    console.log("Password:", password); // Accessing outer function's variable
  };
  return inner;
}
const x = outer(); // outer() execution context is destroyed after this line
if (typeof x === "function") {
  x(); // inner function still has access to userId and password
}
// Output:
// Inside Outer function
// Inside inner function
// User Id: 123
// Password: abc123
```

Key Point: Even though `outer()` has finished executing and its execution context is destroyed, the returned `inner` function still remembers and can access `userId` and `password`.

How Closures Work

The Mechanism Behind Closures

```
function parent() {
  console.log("I am parent function");

  const locker = "10cr"; // This variable will be preserved in closure

  const child = () => {
    console.log("I am child function");
    console.log(locker); // Accessing parent's variable through closure
  };

  return child;
}

const x = parent();
console.log(x); // Logs the child function
x(); // Executes child function with access to 'locker'
```

Closure Memory Model

When JavaScript encounters a closure situation:

1. **JIT Compiler Detection:** The Just-In-Time compiler detects that inner function uses variables from outer scope
2. **Closure Scope Creation:** Creates a special "closure scope" containing only the variables that are actually used by inner function
3. **Memory Preservation:** These variables are NOT garbage collected when outer function ends
4. **Scope Chain:** Inner function's scope chain includes: Own Scope → Closure Scope → Global Scope

```
function createCounter() {  
  let count = 0;           // This will be in closure  
  let unusedVar = "test";  // This will NOT be in closure (not used by inner  
function)  
  
  return function() {  
    count++;               // Uses 'count' from closure  
    console.log("Count:", count);  
  };  
}  
  
const counter = createCounter();  
counter(); // Count: 1  
counter(); // Count: 2
```

Multiple Inner Functions with Closures

```
function parent() {
  const a = 10;

  const inner1 = () => {
    console.log("Inside inner1:", a);
  };

  const inner2 = () => {
    console.log("Inside inner2:", a);
  };

  return [inner1, inner2];
}

// Method 1: Array destructuring
const [x1, x2] = parent();
x1(); // Inside inner1: 10
x2(); // Inside inner2: 10

// Method 2: Array access
const x = parent();
console.log("x =", x); // [function inner1, function inner2]
```

Closures with Function Parameters

```
function parent(a) {
  console.log("Inside Parent function");

  const sum = (b) => {
    console.log("Child function");
    console.log("a =", a); // 'a' comes from parent's parameter (closure)
    console.log("b =", b); // 'b' is child's own parameter
    console.log("Sum:", a + b);
  };

  return sum;
}

const x = parent(10); // Returns sum function with 'a' = 10 in closure
x(20); // Calls sum(20), which has access to 'a' = 10

// Alternative chaining syntax
const result = parent(10)(20); // Same as above, but in one line
```

IIFE with Closures

Example 1: Basic IIFE Closure

```
const increment = (function() {  
    console.log("Inside function 1");  
  
    let count = 0; // Private variable in closure  
  
    return function() {  
        console.log("Inside increment function");  
        count++;  
        console.log("count:", count);  
    };  
})(); // IIFE immediately executed, returns inner function  
  
increment(); // count: 1  
increment(); // count: 2  
increment(); // count: 3
```

Example 2: Double IIFE Execution

```
(function func1() {  
    console.log("Inside function 1");  
  
    let count = 0;  
  
    return function() {  
        console.log("Inside increment function");  
        count++;  
        console.log("count:", count);  
    };  
})(); // Double parentheses: first creates function, second calls returned  
function  
  
// Output:  
// Inside function 1  
// Inside increment function  
// count: 1
```

Example 3: IIFE with Variable Assignment

```
const x = (function func1() {
  console.log("Inside function 1");

  let count = 0;

  return function() {
    console.log("Inside increment function");
    count++;
    console.log("count:", count);
  };
})();

// Each call maintains the same closure
x(); // count: 1
x(); // count: 2
x(); // count: 3
```

Variable Scope Chain in Closures

JavaScript follows this order when looking for variables:

1. **Own Scope:** Variables declared in the current function
2. **Closure Scope:** Variables from outer functions that are referenced
3. **Global Scope:** Variables in global execution context

```
const globalVar = "I'm global";

function outer() {
  const outerVar = "I'm from outer";

  function inner() {
    const innerVar = "I'm from inner";

    console.log(innerVar); // Own scope
    console.log(outerVar); // Closure scope
    console.log(globalVar); // Global scope
  }

  return inner;
}

const closureFunc = outer();
closureFunc();
```

Practical Applications of Closures

1. Data Privacy and Encapsulation

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // Private variable

  return {
    deposit: function(amount) {
      balance += amount;
      console.log(`Deposited: ${amount}. New balance: ${balance}`);
    },

    withdraw: function(amount) {
      if (amount <= balance) {
        balance -= amount;
        console.log(`Withdrawn: ${amount}. New balance: ${balance}`);
      } else {
        console.log("Insufficient funds");
      }
    },

    getBalance: function() {
      return balance;
    }
  };
}

const account = createBankAccount(1000);
account.deposit(500); // Deposited: $500. New balance: $1500
account.withdraw(200); // Withdrawn: $200. New balance: $1300
console.log(account.getBalance()); // 1300

// balance is not directly accessible from outside
console.log(account.balance); // undefined
```

2. Function Factories

```
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

3. Module Pattern

```
const Calculator = (function() {
  let result = 0; // Private variable

  return {
    add: function(x) {
      result += x;
      return this;
    },

    subtract: function(x) {
      result -= x;
      return this;
    },

    multiply: function(x) {
      result *= x;
      return this;
    },

    getResult: function() {
      return result;
    },

    reset: function() {
      result = 0;
      return this;
    }
  };
})();

// Method chaining possible due to returning 'this'
Calculator.add(10).multiply(2).subtract(5);
console.log(Calculator.getResult()); // 15
```

Advantages of Closures

1. **Data Privacy:** Variables in closure are private and can't be accessed directly
2. **State Persistence:** Variables maintain their state between function calls
3. **Factory Functions:** Create multiple instances with their own private state
4. **Callback Functions:** Maintain access to outer scope variables in async operations
5. **Module Pattern:** Create modules with private and public methods

Disadvantages of Closures

1. High Memory Consumption

```
function createHeavyClosure() {  
  const largeArray = new Array(1000000).fill("data"); // Large data structure  
  const anotherLargeArray = new Array(1000000).fill("more data");  
  
  return function(index) {  
    return largeArray[index]; // Only uses largeArray, but both arrays stay in  
memory  
  };  
}  
  
const heavyFunction = createHeavyClosure();  
// Both arrays remain in memory due to closure, even though only one is used
```

2. Multiple Closures Create Separate Memory Spaces

```
function parent() {  
  const sharedData = "This data will be duplicated";  
  
  const inner1 = () => console.log("Inner1:", sharedData);  
  const inner2 = () => console.log("Inner2:", sharedData);  
  
  return [inner1, inner2];  
}  
  
const [func1, func2] = parent();  
// Each function has its own closure with a copy of sharedData  
// Memory usage is doubled
```

3. Memory Leaks in Event Handlers

```
function attachListeners() {  
  const largeData = new Array(1000000).fill("data");  
  
  document.getElementById("button").addEventListener("click", function() {  
    console.log("Button clicked");  
    // largeData remains in memory as long as event listener exists  
  });  
}
```

Best Practices for Closures

1. Only Close Over What You Need

```
// Bad: Closes over entire object
function createProcessor(config) {
  return function(data) {
    return data * config.multiplier; // Only needs multiplier property
  };
}

// Good: Extract only needed value
function createProcessor(config) {
  const multiplier = config.multiplier; // Extract only what's needed

  return function(data) {
    return data * multiplier;
  };
}
```

2. Nullify References When Done

```
function createTemporaryClosure() {
  let tempData = "temporary data";

  return {
    process: function() {
      console.log(tempData);
    },

    cleanup: function() {
      tempData = null; // Help garbage collection
    }
  };
}
```

3. Use WeakMap for Memory-Efficient Closures

```
const privateData = new WeakMap();

function createObject(initialData) {
  const obj = {
    process: function() {
      const data = privateData.get(this);
      console.log(data);
    }
  };

  privateData.set(obj, initialData);
  return obj;
}
```

Common Closure Patterns

1. Counter Pattern

```
function createCounter(start = 0) {
  let count = start;

  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count,
    reset: () => { count = start; }
  };
}
```

2. Once Pattern

```
function once(fn) {
  let called = false;
  let result;

  return function(...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}

const expensiveOperation = once(() => {
  console.log("Expensive operation executed");
  return "result";
});

expensiveOperation(); // Executes and logs
expensiveOperation(); // Returns cached result, no execution
```

3. Memoization Pattern

```
function memoize(fn) {
  const cache = {};

  return function(...args) {
    const key = JSON.stringify(args);

    if (cache[key]) {
      console.log("Cache hit");
      return cache[key];
    }

    console.log("Computing result");
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}

const fibonacciMemoized = memoize(function(n) {
  if (n <= 1) return n;
  return fibonacciMemoized(n - 1) + fibonacciMemoized(n - 2);
});
```

Debugging Closures

Use browser developer tools to inspect closures:

```
function debugClosure() {  
  const secretValue = "debug me";  
  let counter = 0;  
  
  return function() {  
    counter++;  
    debugger; // Set breakpoint here  
    console.log(secretValue, counter);  
  };  
}  
  
const debugFunc = debugClosure();  
debugFunc(); // Check "Scope" panel in DevTools to see closure variables
```

Closures are one of JavaScript's most powerful features, enabling elegant solutions for data privacy, state management, and functional programming patterns. Understanding closures deeply is crucial for mastering JavaScript and writing efficient, maintainable code.